

# Estudio comparativo de heurísticas de construcción de un BVH para trazado de rayos en CPU

CS3014 – Estructura de Datos Avanzados (2026-1)

Autor 1, Autor 2, Autor 3

## Resumen

Un trazador de rayos ingenuo prueba cada rayo contra todos los triángulos ( $O(N)$  por rayo). Una jerarquía de volúmenes envolventes (BVH) reduce este costo a  $\sim O(\log N)$  agrupando los triángulos en un árbol de cajas alineadas a los ejes (AABB). En este proyecto implementamos un ray tracer por CPU con BVH y comparamos **tres heurísticas de construcción** (SAH, median-split y Morton/LBVH) y una **referencia externa** (Intel Embree) sobre la misma geometría, midiendo tiempo de construcción, calidad del árbol (nodos visitados por rayo) y costo de recorrido. Los resultados confirman el tradeoff clásico: SAH produce el árbol de mejor calidad, Morton se construye más rápido pero da el peor árbol, y los kernels SIMD de Embree recorren  $\sim 3\times$  más rápido que nuestra mejor SAH.

## 1. Introducción

El trazado de rayos resuelve la visibilidad simulando el camino de la luz. Por cada píxel se emite al menos un rayo; con sombras y reflejos, varios. Para  $P$  píxeles y  $N$  triángulos, la fuerza bruta cuesta  $O(P \cdot N)$  por frame, inmanejable para escenas de decenas de miles de triángulos. La aceleración consiste en una estructura que descarte grandes grupos de triángulos con pruebas baratas. El BVH es la opción dominante por su memoria  $O(N)$  y su robustez.

## 2. El BVH y la heurística SAH

Un BVH agrupa los triángulos en un árbol binario de AABB anidadas: la raíz encierra toda la escena; cada nodo interior particiona sus triángulos entre dos hijos; las hojas guardan unos pocos triángulos. Un rayo prueba las cajas baratas primero y solo desciende por las que golpea, saltando subárboles enteros.

La **heurística de área de superficie (SAH)** estima el costo esperado de un corte a partir del área de cada caja hijo (un rayo es más probable de entrar en una caja más grande):

$$C_{\text{split}} = 1 + \frac{A_L n_L + A_R n_R}{A_{\text{padre}}}$$

y se elige el corte de menor costo, comparándolo contra no partir (hoja). Para no evaluar todos los cortes, se agrupan los centroides en 12 *bins* por eje (*binned SAH*).

## 3. Estrategias de construcción comparadas

Implementamos tres estrategias, seleccionables en tiempo de ejecución:

- **SAH** (*binned*): la del apartado anterior. Mejor calidad, construcción  $O(N \log N)$ .
- **Median**: parte por la mediana de los centroides en el eje más largo. Sencilla y rápida, calidad intermedia.
- **Morton/LBVH**: ordena todos los triángulos por su código de Morton (Z-order) del centroide y luego parte por el índice medio. Construcción muy rápida, calidad menor.

El recorrido es iterativo con pila fija (profundidad  $\leq 60$ ), intersección rayo-AABB por *slabs* y rayo-triángulo por Möller-Trumbore.

## 4. Metodología experimental

Escena: ciudad procedural de edificios (fachadas con ventanas) en tres tamaños (*Small/Medium/Large*, ~16k–33k triángulos). Para cada estrategia medimos, sobre el mismo conjunto de rayos primarios:

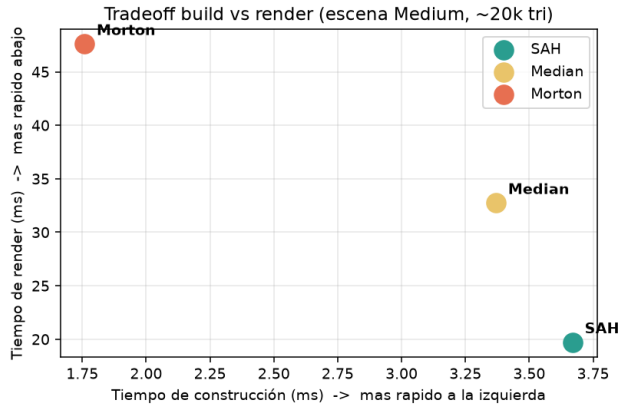
- **build**: tiempo de construir el árbol.
- **nodos/ray**: nodos del BVH probados por rayo (calidad del árbol; independiente del hardware  $\Rightarrow$  comparable entre máquinas).
- **traversal**: tiempo de intersección pura, *single-threaded* y sin sombreado  $\Rightarrow$  comparación justa con Embree.

Como referencia externa usamos **Intel Embree 4** sobre los mismos triángulos (mismos rayos, `rtcIntersect1`). Plataforma: Apple Silicon (ARM64, NEON). Código: `-O2 -march=native`.

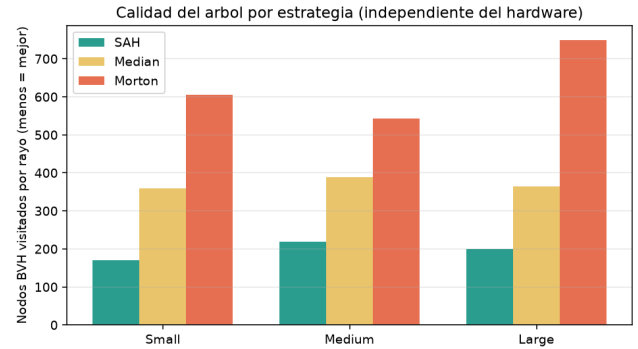
## 5. Resultados

Cuadro 1: Escena *Medium* (~19,750 triángulos).

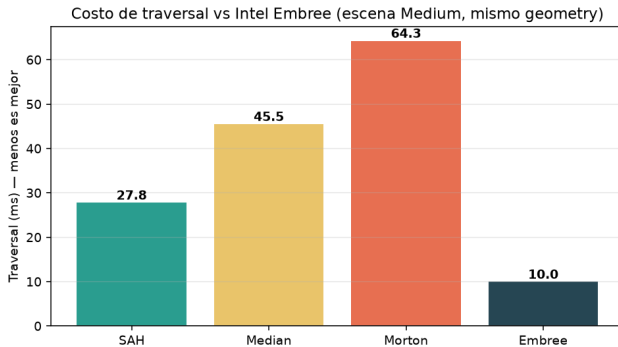
| Estrategia | build (ms)  | render (ms) | traversal (ms) | nodos/ray  |
|------------|-------------|-------------|----------------|------------|
| SAH        | 3.67        | <b>19.7</b> | 27.8           | <b>218</b> |
| Median     | 3.37        | 32.7        | 45.5           | 388        |
| Morton     | <b>1.76</b> | 47.6        | 64.3           | 542        |
| Embree     | 2.31        | —           | <b>9.96</b>    | —          |



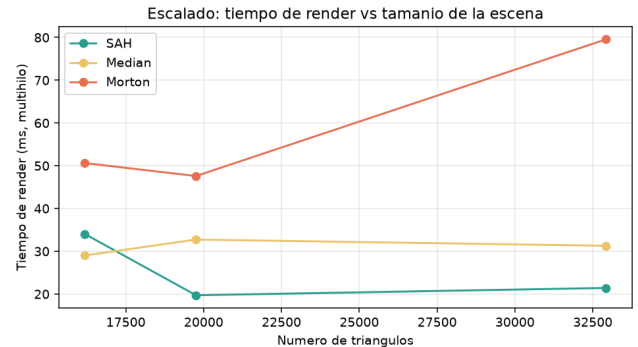
Tradeoff build vs. render: SAH más a la derecha pero más abajo.



Calidad: nodos/ray (menos mejor). SAH gana.



Traversal vs. Embree (*Medium*): ~3× más rápido.



Escalado: render vs. número de triángulos.

## 6. Discusión

Entre nuestras heurísticas, los resultados confirman el tradeoff build-calidad: Morton se construye  $\sim 2\times$  más rápido que SAH pero su árbol es  $\sim 2,5\times$  peor (542 vs. 218 nodos/ray), lo que lastra su render. SAH es el ganador global en tiempo de render.

Frente a Embree, la brecha en *traversal* ( $\sim 3\times$ ) NO se explica por la calidad del árbol (nuestra SAH es competitiva) sino por la **ingeniería del kernel**: Embree recorre el árbol con SIMD (NEON) procesa varios nodos/rayos a la vez, usa nodos de 4/8 vías (MBVH) y un *layout* alineado a línea de caché. Es decir, nuestra *estructura de datos* es razonable; la diferencia está en la *implementación* del recorrido —una distinción relevante para una materia de estructuras de datos.

## 7. Limitaciones y trabajo futuro

La rúbrica valora las continuaciones; para cerrar la brecha con Embree:

- Recorrido **SIMD/NEON**: procesar varios rayos (o varios hijos) por instrucción.
- **MBVH**: nodos de 4 vías en vez de binarios  $\Rightarrow$  menos nodos visitados.
- **Layout SoA** y nodos cuantizados para mejor uso de caché.
- Construcción paralela y un modo de comparación de Morton con BVH espacial (SBVH) para geometría que se intersecta.

## 8. Conclusiones y contribuciones

Implementamos un ray tracer con BVH y un *benchmark* reproducible (`--bench`) que compara heurísticas y la referencia Embree. El estudio muestra empíricamente que **SAH** ofrece el mejor árbol, que **Morton** intercambia calidad por velocidad de construcción, y cuantifica la diferencia con el estado del arte industrial.

**Contribuciones por autor.** (a completar) Autor 1: BVH y heurísticas; Autor 2: escena y renderer; Autor 3: métricas, *benchmark* y Embree.

**Repositorio y demo.** Código, datos (`docs/benchmark.csv`), gráficas (`docs/fig_*.png`) y script (`docs/plot.py`) en el repositorio del proyecto; video demo y diapositivas en los enlaces adjuntos.